# LAB # 03

# LOGISTIC REGRESSION

## Import Required Libraries · Load Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
cancer_data = load_breast_cancer()

df = pd.DataFrame(cancer_data.data, columns=cancer_data.feature_names)
df['target'] = cancer_data.target
```

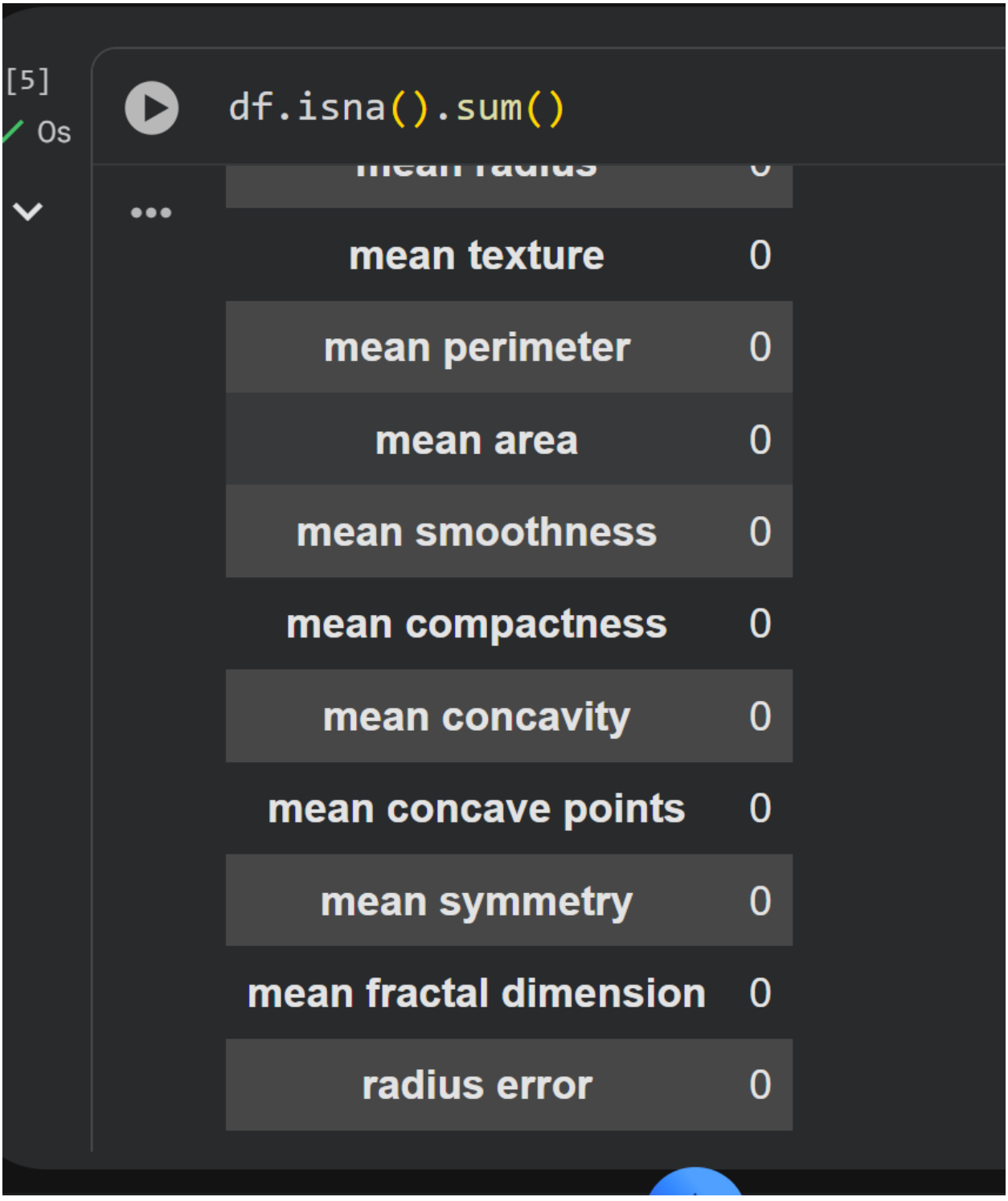
**• Use a dataset Breast Cancer Dataset from sklearn.datasets. · Exploratory Data Analysis (EDA) • Display first few rows of the dataset.**

```
df.head()
```

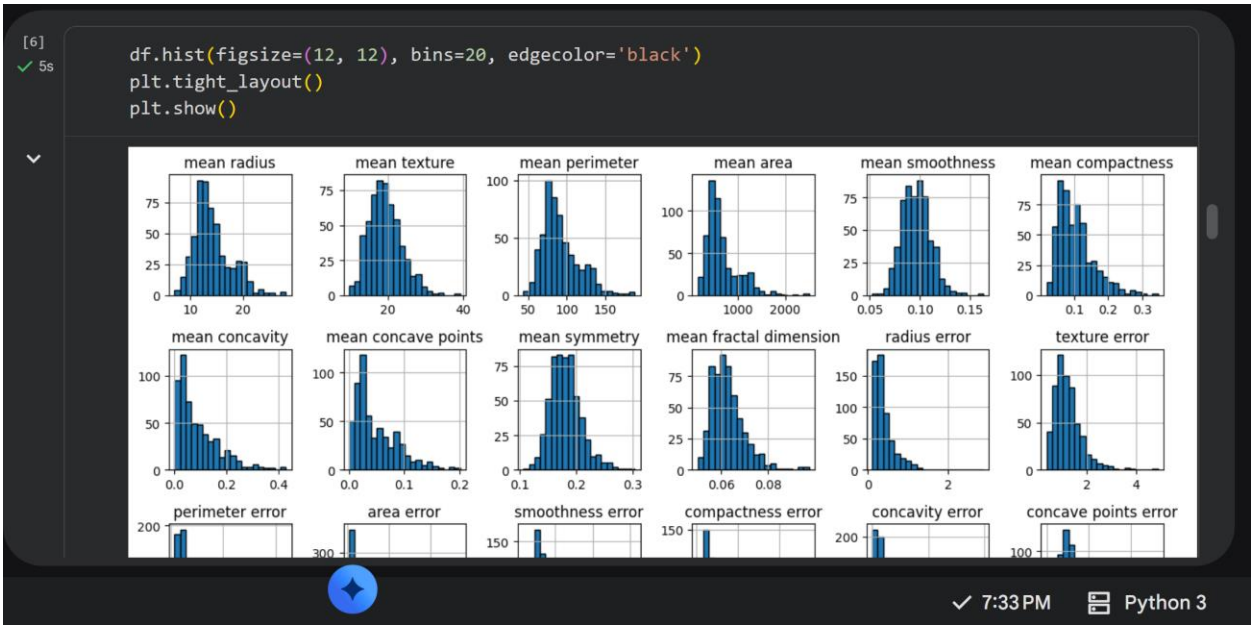
| mean adius | mean texture | mean perimeter | mean area | mean smoothness | mean compactness | mean concavity | mean concave points | mean symmetry | mean fractal dimension |
|---|---|---|---|---|---|---|---|---|---|
| 17.99 | 10.38 | 122.80 | 1001.0 | 0.11840 | 0.27760 | 0.3001 | 0.14710 | 0.2419 | 0.07871 |
| 20.57 | 17.77 | 132.90 | 1326.0 | 0.08474 | 0.07864 | 0.0869 | 0.07017 | 0.1812 | 0.05667 |
| 19.69 | 21.25 | 130.00 | 1203.0 | 0.10960 | 0.15990 | 0.1974 | 0.12790 | 0.2069 | 0.05999 |
| 11.42 | 20.38 | 77.58 | 386.1 | 0.14250 | 0.28390 | 0.2414 | 0.10520 | 0.2597 | 0.09744 |
| 20.29 | 14.34 | 135.10 | 1297.0 | 0.10030 | 0.13280 | 0.1980 | 0.10430 | 0.1809 | 0.05883 |

s × 31 columns

- **Check for missing values**



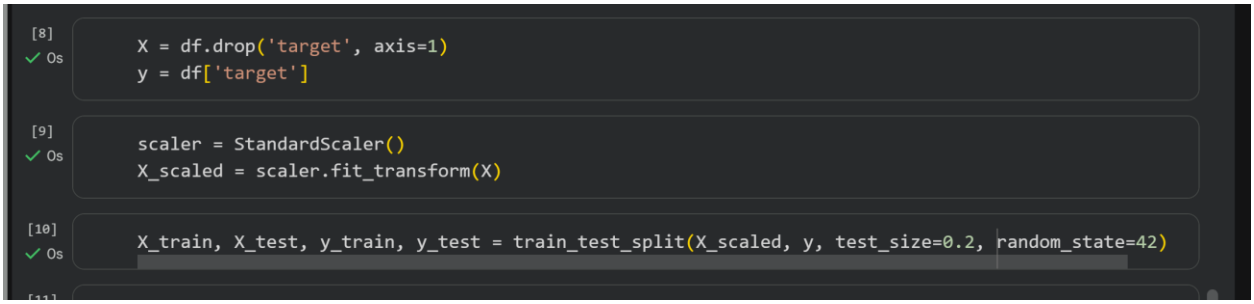
- **Plot histograms or distribution of features.**




- **Check class distribution (malignant vs. benign).**

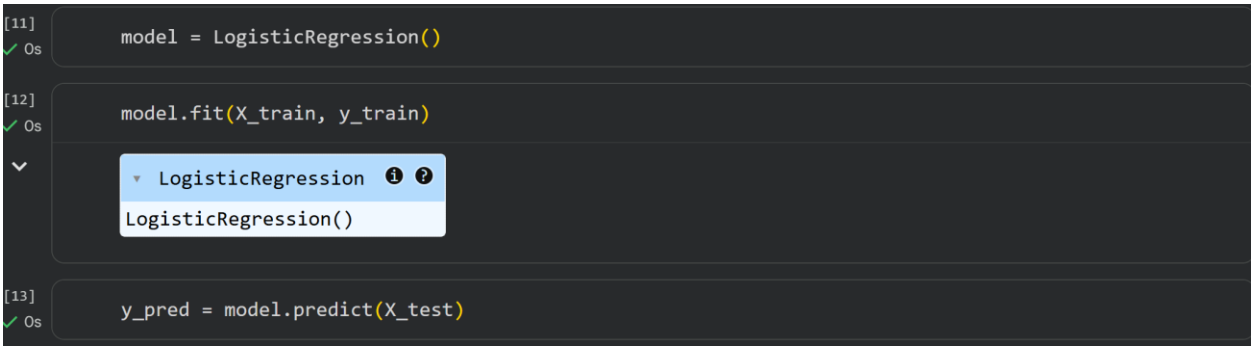


· Data Preprocessing • Split features and labels. • Standardize features using StandardScaler. • Train-test split




## Model Implementation

- Use sklearn.linear_model.LogisticRegression.
- Train the model on the training data.
- Predict on the test data.




· Evaluation

- Accuracy score.

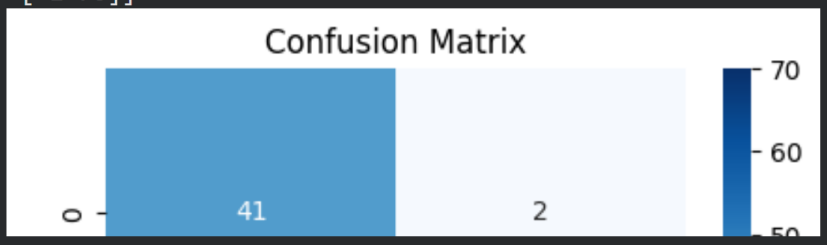


- **Confusion matrix.**

```
[15]
✓ 0s
```

```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

```
Confusion Matrix:
[[41  2]
 [ 1 70]]
```




- **Precision, Recall, F1-score.**

```
[16]
✓ 0s
```

```
report = classification_report(y_test, y_pred, target_names=['Malignant', 'Benign'])
print("Classification Report:")
print(report)
```

```
Classification Report:
              precision    recall  f1-score   support

   Malignant       0.98      0.95      0.96        43
      Benign       0.97      0.99      0.98        71

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114
```